

Design Report: Concurrent Task Dispatcher

=====

Overview

This project implements a concurrent task dispatcher in Rust simulating an OS-style scheduler. A generator thread produces 1000 tasks at fixed 20 ms intervals – 70% IO-bound (10% CPU load, 200 ms) and 30% CPU-bound (35% CPU load, 200 ms). A manager-queue thread enforces a global 100% CPU cap, routes tasks through typed queues, and dispatches to eight worker threads. A monitor thread samples system state every 10 ms. Two scheduling policies are compared: FIFO and a batch-aware Optimized policy.

Threads and Components (11 total)

Main wires everything, joins threads in order, and prints the comparison.

Generator sleeps 20 ms between tasks and sends each down a channel. Dropping the sender when finished is the zero-cost shutdown signal to the manager.

Manager-queue (dispatcher) is the scheduler. It owns both task queues outright – no lock needed since no other thread touches them. Each iteration it ingests arrivals, collects completion reports, and dispatches tasks to idle workers within the CPU cap. A local `worker_cpu[]` array tracks each busy worker's load so the manager knows exactly how much to subtract on each completion.

Workers (x8) block on a personal channel. Both task types sleep 200 ms to simulate execution. On completion they send a `CompletionReport` back and immediately wait for the next task. They exit cleanly on receiving `None`.

Monitor reads two atomics (global CPU%, active workers) every 10 ms and stores snapshots into shared Metrics. It holds no locks and cannot interfere with dispatch.

Shared State and Synchronization

Task queues are local to the manager – no synchronization needed.

`Arc<Mutex<Metrics>>` is the only shared mutable state. The manager writes one record per completion (1000 total); main reads the result after joining. Lock contention is negligible.

`Arc<AtomicU32>` (CPU%) and `Arc<AtomicUsize>` (active workers) are written only by the manager and read by the monitor with relaxed ordering. An approximate value one sample behind is acceptable for a progress display.

Channels carry all task data: `generator→manager`, `manager→each worker` (`Option<Task>`, with `None` as shutdown), and `workers→manager` (shared mpsc for `CompletionReports`). Channels were chosen over shared queues because they transfer ownership cleanly, buffer across rate differences, and the sender-drop provides a built-in shutdown notification.

Global CPU Cap

Each task carries a `cpu_percent` field (CPU=35, IO=10). The manager keeps a local `current_cpu` counter, checks `current_cpu + task.cpu_percent <= 100` before

every dispatch, increments on dispatch, and decrements on completion. This hard cap prevents the system from scheduling more concurrent load than the simulated machine can handle.

Scheduling Policies

FIFO places all tasks into a single arrival-order queue. The manager checks the front task: if it fits within the cap and a worker is idle, it dispatches. If it does not fit, dispatch halts – head-of-line blocking. An IO task queued behind a CPU task cannot run even when IO would fit within the cap.

Optimized maintains separate CPU and IO queues. Each dispatch cycle it computes the CPU-task ratio in the queues and selects a target batch based on the LP formulation from the spec:

Option 1: 2 CPU + 3 IO = 100% CPU, 5 workers
Option 2: 1 CPU + 6 IO = 95% CPU, 7 workers (best throughput)
Option 3: 0 CPU + 8 IO = 80% CPU, 8 workers

When `cpu_ratio` $\geq$ 30%: target 2 CPU per cycle (Option 1).

When `cpu_ratio` 12-30%: target 1 CPU + fill IO (Option 2).

When `cpu_ratio` < 12%: pure IO (Option 3).

IO tasks can always dispatch when headroom allows regardless of the CPU queue, eliminating head-of-line blocking.

Experiment Results

Both simulations used the same seed-42 workload: 1000 tasks, 70% IO / 30% CPU.

Metric	FIFO	Optimized
Total runtime	39 210 ms	41 910 ms
Avg wait time	9 206 ms	5 452 ms
Avg turnaround	9 406 ms	5 652 ms
Avg CPU usage	89%	83%
Avg active workers	5/8	4/8

FIFO achieves better throughput (39.2 s vs 41.9 s) because with 70% IO tasks it naturally dispatches long IO runs that keep CPU near 89%. When 8 IO tasks run simultaneously that is 80% CPU; adding the occasional CPU task pushes toward 100%, efficiently using the cap. The complex batch selection in Optimized tends to lock in 5-worker configurations (Option 1) during peak load, slightly reducing parallelism.

Optimized achieves a 41% reduction in average wait time (9.2 s $\rightarrow$ 5.5 s) by eliminating head-of-line blocking. IO tasks are never delayed behind a CPU task that cannot yet run. This is the classic throughput-vs-latency trade-off: FIFO wins on total completion time; Optimized wins on individual responsiveness.

Concurrency Bug Encountered

The original termination condition used `idle.is_empty()`. This fires as soon as all workers have been dispatched – including while they are still executing. The manager would exit, send None to workers, and drop the completion channel before final reports arrived. Those reports were silently lost, producing a short metrics count.

Fix: changed to `idle.len() == num_workers`. This is only true when every worker has finished, sent its report, and been moved back into the idle set. Dispatched does not mean done.

Where Problems Remain

FIFO: IO tasks queued behind CPU tasks face unbounded delay when CPU tasks arrive continuously. A task could wait for several consecutive CPU-task completions before the head clears.

Optimized: CPU tasks accumulate if the system is IO-saturated. With at most 2 CPU tasks running simultaneously ($2 \times 35 = 70\%$), a large CPU backlog drains slowly and extends the tail of the simulation.

Both: workers sit idle when CPU usage is between 66% and 99% because no task type fits cleanly. Work stealing across task types (within the cap) would improve utilization but would require relaxing the typed-queue separation.

Lessons Learned

Simple policies can beat complex ones for the right workload. With 70% IO, FIFO naturally selects efficient batches without explicit logic. The Optimized policy's overhead and structure do not always align with what the workload actually needs – an important reminder that optimization requires knowing the workload first.

Throughput and latency are different goals. Optimized cut average wait by 41% at a 7% throughput cost. Which matters more depends on the application, not the scheduler.

Termination correctness requires tracking completion, not dispatch.
`idle.len() == num_workers` is the right quiescence condition.